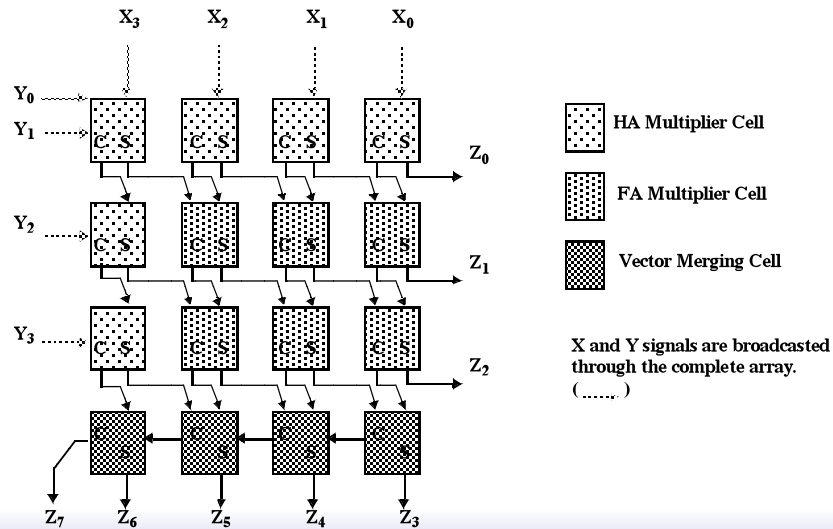


Multiplier Floorplan

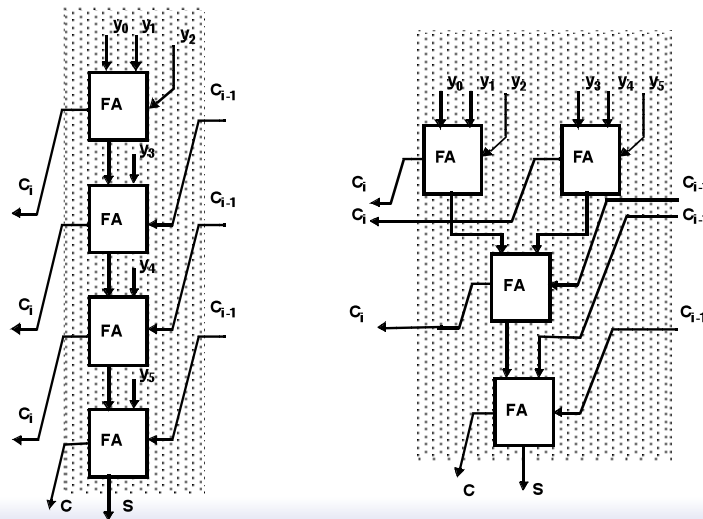


EECS141

Lecture #24

11

Wallace-Tree Multiplier

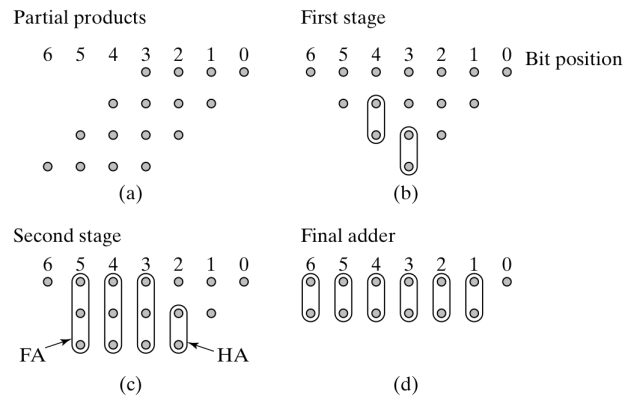


EECS141

Lecture #24

12

Wallace-Tree Multiplier

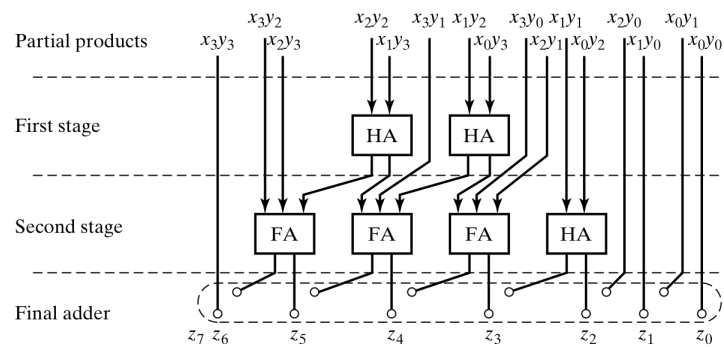


EECS141

Lecture #24

13

Wallace-Tree Multiplier



EECS141

Lecture #24

14

Multipliers – Summary

- Optimization constraints different than in binary adder
 - Once again:
 - Need to identify critical path
 - And find ways to use parallelism to reduce it
- Other possible techniques
 - Logarithmic versus linear (Wallace Tree Mult)
 - Data encoding (Booth)
 - Pipelining

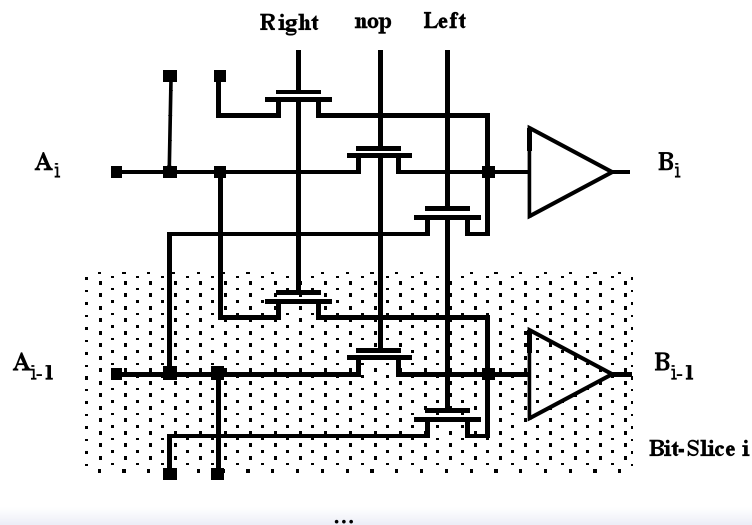
First glimpse at system level optimization

EECS141

Lecture #24

15

The Binary Shifter

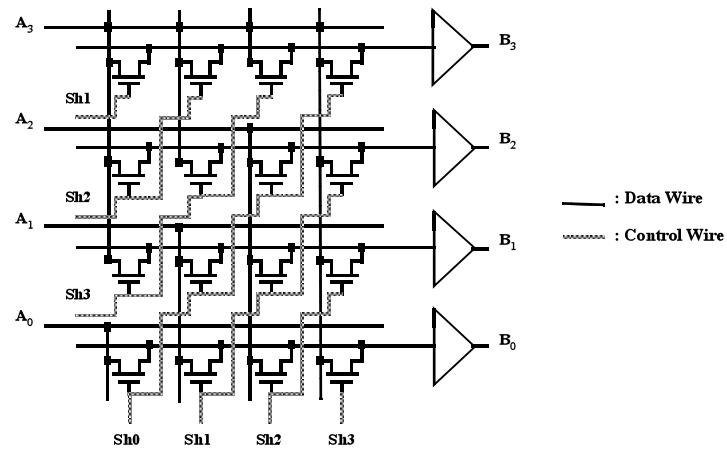


EECS141

Lecture #24

16

The Barrel Shifter



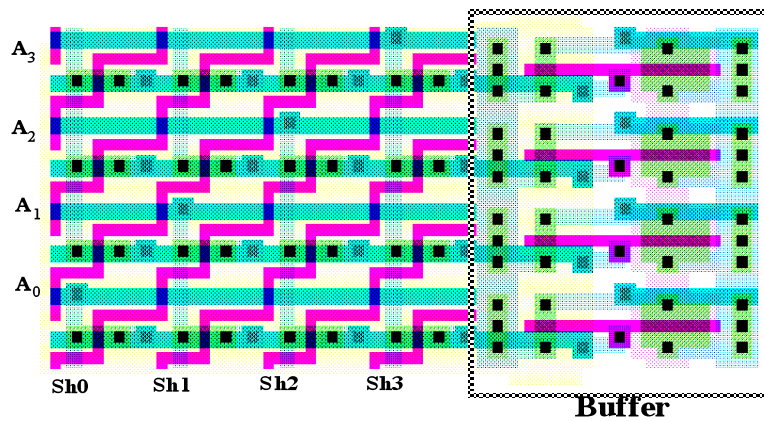
Area Dominated by Wiring

EECS141

Lecture #24

17

4x4 Barrel Shifter



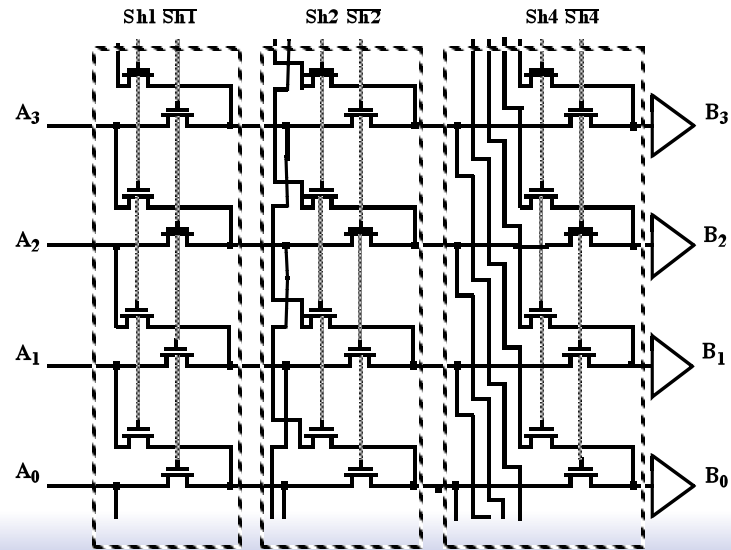
$\text{Width}_{\text{barrel}} \sim 2 p_m M$

EECS141

Lecture #24

18

Logarithmic Shifter

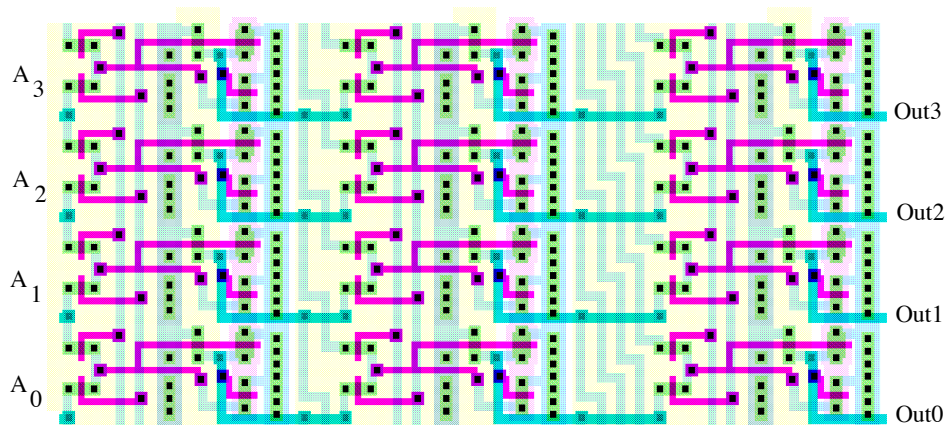


EECS141

Lecture #24

19

0-7 bit Logarithmic Shifter



$$width_{\log} \approx p_m \cdot \left\lfloor 2K + (1 + 2 + \dots + 2^{K-1}) \right\rfloor = p_m \cdot (2^K + 2K - 1)$$

EECS141

Lecture #24

20

Arithmetic - Summary

EECS141

Lecture #24

21



Semiconductor Memory

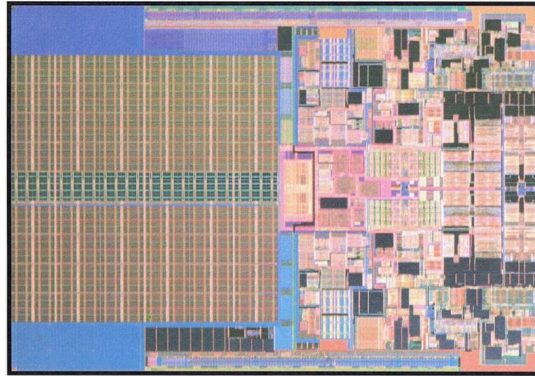
EECS141

Lecture #24

22²²

Why Memory?

Intel 45nm Core 2



EECS141

Lecture #24

23

Semiconductor Memory Classification

Read-Write Memory		Non-Volatile Read-Write Memory	Read-Only Memory
Random Access	Non-Random Access	EPROM E ² PROM FLASH	Mask-Programmed Programmable (PROM)
SRAM DRAM	FIFO LIFO Shift Register CAM		

EECS141

Lecture #24

24

Random Access Memories (RAM)

❑ STATIC (SRAM)

Data stored as long as supply is applied
Larger (6 transistors/cell)
Fast
Differential (usually)

❑ DYNAMIC (DRAM)

Periodic refresh required
Smaller (1-3 transistors/cell)
Slower
Single Ended

EECS141

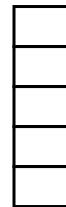
Lecture #24

25²⁵

Random Access Chip Architecture

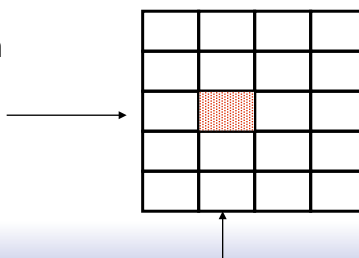
❑ Conceptual: linear array

- Each box holds some data
- But this does not lead to a nice layout shape
- Too long and skinny



❑ Create a 2-D array

- Decode Row and Column address to get data

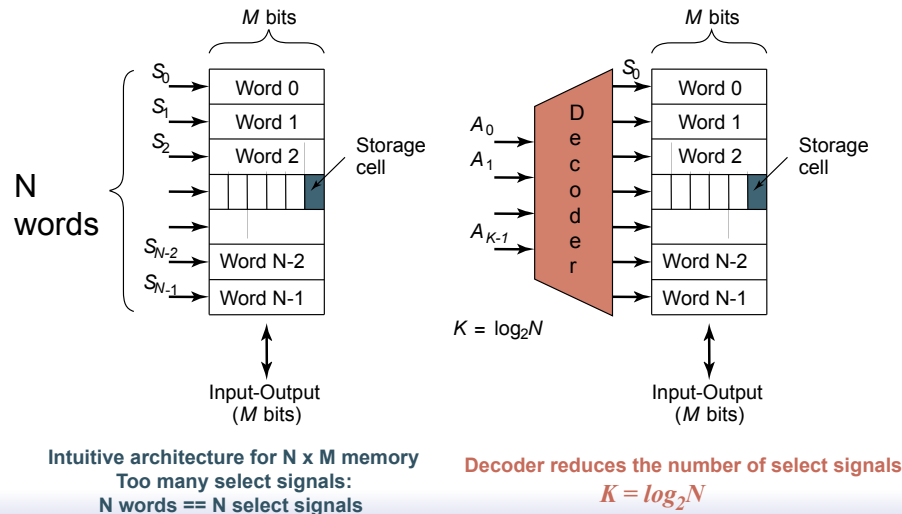


EECS141

Lecture #24

26

Memory Architecture: Decoders



EECS141

Lecture #24

27

Basic Memory Array

CORE:

- keep square within a 2:1 ratio
- rows are **word lines**
- columns are **bit lines**
- data in and out on columns

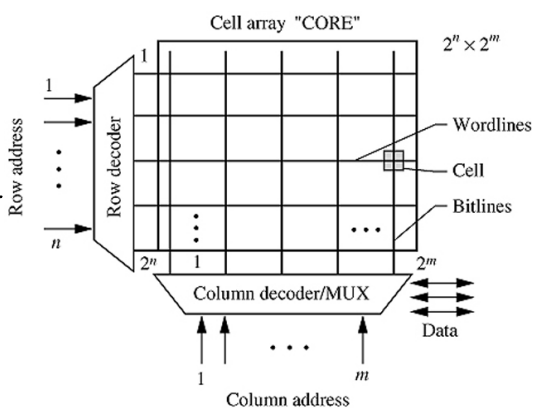
DECODERS:

- needed to reduce total number of pins; $N+M$ address lines for 2^{N+M} bits of storage

Ex: if $N+M=20 \rightarrow 2^{20} = 1\text{Mb}$

MULTIPLEXING:

- used to select one or more columns for input or output of data



EECS141

Lecture #24

28

Row Decoders

Collection of 2^M complex logic gates
Organized in regular and dense fashion

(N)AND Decoder

$$WL_0 = \overline{A_0} \overline{A_1} \overline{A_2} \overline{A_3} \overline{A_4} \overline{A_5} \overline{A_6} \overline{A_7} \overline{A_8} \overline{A_9}$$

$$WL_{511} = A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A_9$$

NOR Decoder

$$WL_0 = \overline{A_0 + A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7 + A_8 + A_9}$$

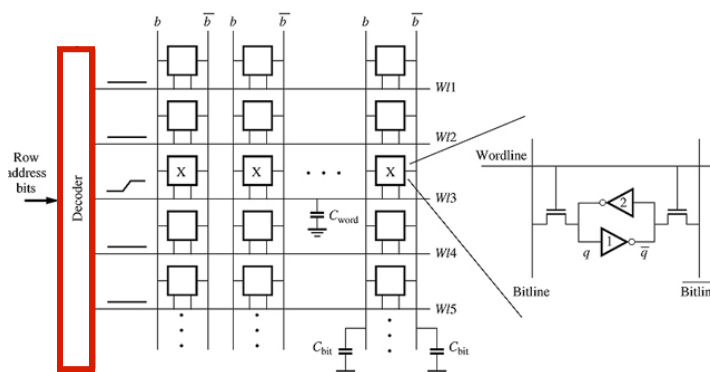
$$WL_{511} = \overline{A_0} + \overline{A_1} + \overline{A_2} + \overline{A_3} + \overline{A_4} + \overline{A_5} + \overline{A_6} + \overline{A_7} + \overline{A_8} + A_9$$

EECS141

Lecture #24

29

Decoder Design Example



- Look at decoder for 256x256 memory block (8KBytes)

EECS141

Lecture #24

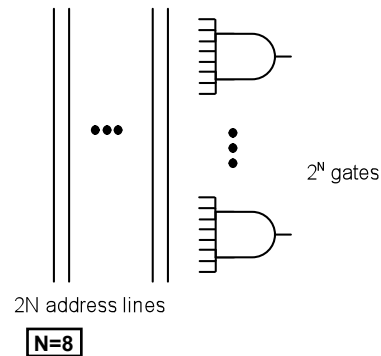
30

Problem Setup

- Goal: Build fastest possible decoder with static CMOS logic

- What we know

- Basically need 256 AND gates, each one of them drives one word line



EECS141

Lecture #24

31

Problem Setup (1)

- Each word line has 256 cells connected to it
- Total output load is $256 \cdot C_{\text{cell}} + C_{\text{wire}}$
- Assume that decoder input capacitance is $C_{\text{address}} = 4 \cdot C_{\text{cell}}$
- Each address drives $2^8/2$ AND gates
 - A0 drives $1/2$ of the gates, A0_b the other $1/2$ of the gates
- Neglecting C_{wire} , the fan-out on each one of the 16 address wires is:

$$BF = \frac{C_{\text{load}}}{C_{\text{in}}} = \frac{(2^8/2)(2^8 C_{\text{cell}})}{4C_{\text{cell}}} = 2^{13}$$

EECS141

Lecture #24

32

Decoder Fan-Out

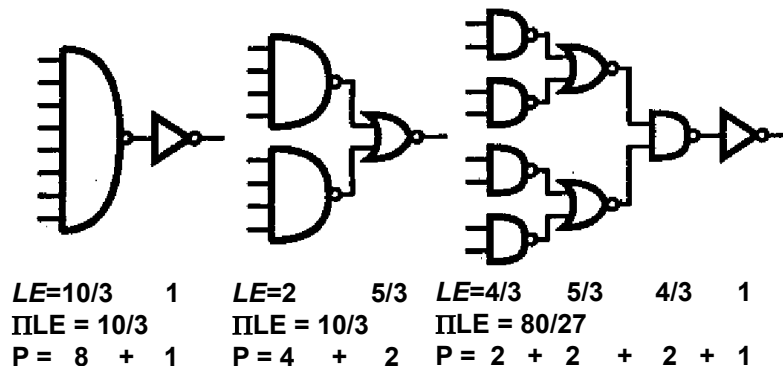
- FB of at least 2^{13} means that we will want to use more than $\log_4(2^{13}) = 6.5$ stages to implement the AND8
- Need many stages anyways
 - So what is the best way to implement the AND gate?
 - Will see next that it's the one with the most stages and least complicated gates

EECS141

Lecture #24

33

8-Input AND

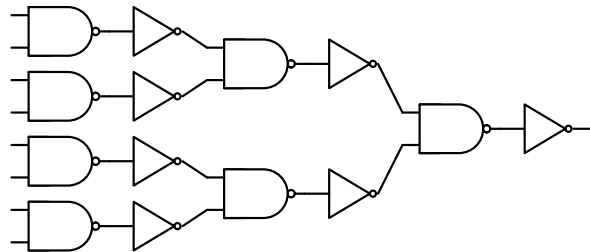


EECS141

Lecture #24

34

8-Input AND



- Using 2-input NAND gates
 - 8-input gate takes 6 stages
- Total LE is $(4/3)^3 \approx 2.4$
- So PE is $2.4 \cdot 2^{13}$ – optimal N of ~ 7.1

EECS141

Lecture #24

35

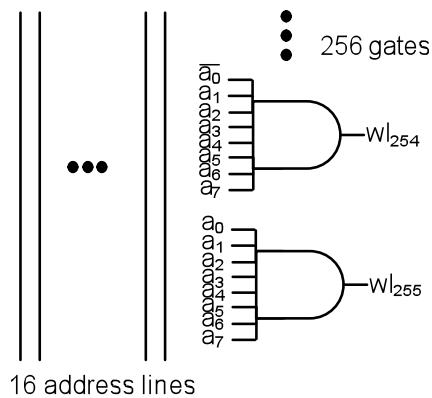
Decoder So Far

- 256 8-input AND gates

- Each built out of tree of NAND gates and inverters

- Issue:

- Every address line has to drive 128 gates (and wire) right away
- Can't build gates small enough - Forces us to add buffers just to drive address inputs

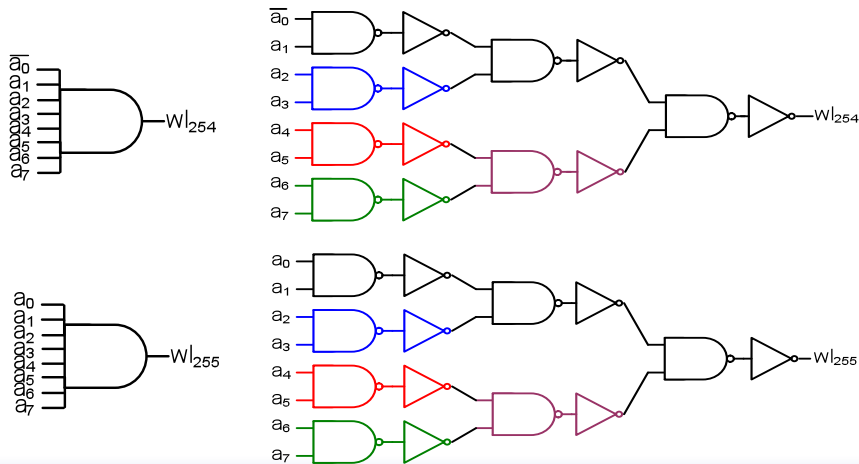


EECS141

Lecture #24

36

Look Inside Each AND8 Gate



EECS141

Lecture #24

37

Predecoders

- Use a single gate for each of the shared terms
 - E.g., from A_0 , $\overline{A_0}$, A_1 , and $\overline{A_1}$, generate four signals: A_0A_1 , $\overline{A_0}A_1$, $A_0\overline{A_1}$, $\overline{A_0}\overline{A_1}$
- In other words, we are decoding smaller groups of address bits first
 - And using the “predecoded” outputs to do the rest of the decoding

EECS141

Lecture #24

38

Predecoder and Decoder

